

# Deployment Plan

## From Prototype to Supabase Migration

### 1. Scope and Objectives

This plan governs the transition of the **Job Launch** platform from its current prototype state to a final, hosted submission.

- **Current State (Prototype):** Built on Core PHP, MySQL for the database, managing sessions, and utilizing static/HTML hybrid views primarily located in the Users/ directory.
- **Target State (Final Submission):** The system's backend architecture will migrate to **Supabase** (Backend-as-a-Service), replacing local MySQL with PostgreSQL, Supabase Authentication, and Storage.

#### Objectives

1. Ensure repeatable and verified deployments for both the prototype and the Supabase final submission.
2. Implement robust secret protection mechanisms to prevent credential leaks.
3. Establish clear rollback procedures and verification steps to minimize deployment risk.

### 2. Environments and Architecture

#### Environments

- **Local Development:** Uses XAMPP, WAMP, or a PHP built-in server alongside a local MySQL instance.
- **Final Hosted:** Supabase handles the Database, Authentication, and Storage. The PHP frontend will be served via a hosting solution (or potentially reimplemented using a Supabase client SDK).

#### Architecture Snapshot

- **Frontend:** HTML-PHP hybrid pages (index.php and pages in Users/).
- **Prototype Backend:** PHP scripts in backendwithphp/ use mysqli for direct MySQL interaction.
- **Final Backend:** PHP (or JS) will communicate with Supabase (PostgreSQL, Auth, Storage) via client SDK or REST APIs. Direct MySQL usage will be eliminated.

### 3. Prerequisites

Before deployment, the following components and access must be secured:

## Group-8

- **Runtime:** PHP 8.x.
- **Dependency Manager:** Composer (if needed for dependencies).
- **Database Access:** Local MySQL instance (Prototype) or API access to a dedicated Supabase project (Final).
- **Server Setup:** Web server root configured to point to the project folder.
- **Supabase Access:** Project dashboard access and the Anon and Service Role keys for final deployment.

### 4. Deployment Steps: Phase 1 (Prototype with MySQL)

1. **Repository Setup:** Clone the project repository to the target server.
2. **Database Configuration:** Create a MySQL database named `job_launch`. Import the schema (tables for users, company, job\_seeker, jobs, etc.).
3. **Connection Update:** Configure `backendwithphp/db_conection.php` with the correct host, username, password, and database name.
4. **File Placement:** Place the code under the web root (e.g., `htdocs/job-web(prototype)`), ensuring `index.php` is the entry point.
5. **Smoke Testing:** Verify core functionality: registration, login, job posting (employer), job application (seeker), and job listings rendering.

### 5. Deployment Steps: Phase 2 (Final with Supabase/PostgreSQL)

1. **Project Initialization:** Create a new Supabase project. Securely record the **API URL** and **Anon/Service Keys**.
2. **Table Provisioning:** Provision necessary tables (e.g., `company`, `job_seeker`, `jobs`, `applications`) in Supabase, aligning columns with the planned schema and defining primary/foreign keys.
3. **Secret Management:** Create a `.env` file for local testing and use environment variables/host configuration to store the Supabase URL and keys on the server. **Do not commit secrets to VCS.**
4. **Backend Migration:** Update the PHP backend to use the Supabase client SDK or REST APIs for all CRUD operations and authentication. **Remove all direct mysql database calls.**
5. **Hosting:** Deploy the static/HTML-PHP pages to the host, ensuring they correctly reference the Supabase configuration via environment variables.
6. **End-to-End Verification:** Conduct a final smoke test in the hosted environment, checking authentication, posting, applying, and listing workflows using Supabase.

### 6. Configuration Management

- **Secrets:** All sensitive credentials (DB passwords, Supabase keys) must be kept out of version control and managed via environment variables or a host dashboard.
- **Environment Files:** Use the `.env` / `.env.example` pattern.
- **Database Migrations:** Document all schema changes (SQL scripts for MySQL prototype; migration notes for Supabase/PostgreSQL).

#### Group-8

- **Versioning:** Tag releases appropriately (e.g., v0.1-prototype, v1.0-supabase).
- **Backups:** Enable automated daily backups for Supabase. For the MySQL prototype, export SQL files before all major deployments.

## 7. Rollback and Monitoring

### Rollback Strategy

- **Prototype:** Revert the codebase to the previous tagged version and restore the corresponding database backup.
- **Supabase:** Redeploy the previous application version and restore the database using Supabase's backup or point-in-time recovery features.

### Monitoring and Logs

- **Prototype:** Monitor web server access logs and PHP error logs.
- **Supabase:** Use the Supabase dashboard for database and auth logs. Monitor hosting logs for frontend and client-side backend errors.